

Teacher Module — Priority Rules

Priority Rules Before Implementation

Before starting any development, follow these rules strictly:

1. **No placeholder data**
 - Remove any usage of `Math.random()` or mock values.
 - All metrics must come from **real database queries**.
2. **Database-driven architecture**
 - Every feature must read/write from the database.
 - No temporary in-memory logic.
3. **Audit logging**
 - Every action must be recorded in `audit_logs`.
4. **Mobile-first UI**
 - Teacher interface must prioritize **mobile experience**.
5. **Backward compatibility**
 - Do not break existing APIs or working flows.

Phase 1 — Data Integrity & Core Missing Structures

Start this phase immediately

Goal: Fix structural gaps that affect data integrity.

1. Fix Placeholder Data in My Classes

Replace all placeholder logic in:

`TeacherClassesPage.jsx`

Replace:

`Math.random()`

with real metrics from database:

Required metrics:

- class attendance rate
- participation rate
- average performance
- next session time

Create a backend endpoint if needed:

GET /teacher/{id}/class-metrics

2. Create Skills System

Currently behaviour types exist but **skills do not**.

You must implement:

New Collections

skills_types

student_skills

skills_types

Fields:

- id
- name
- description
- category
- created_at

Example seed data:

- Leadership
- Cooperation
- Initiative
- Creativity
- Critical Thinking
- Responsibility

student_skills

Fields:

- id
- student_id
- class_id
- session_id
- skill_type_id
- recorded_by_teacher
- timestamp

3. Extend Session Engine

Add a new function to:

TeacherSessionEngine

record_skill()

New API endpoint:

POST /session/{id}/skill

Request body:

```
{  
  student_id,  
  skill_type_id,  
  notes  
}
```

This action must:

- store skill record
- update student session metrics
- log event in audit log

4. Add Skill Recording UI

Inside:

SessionTeachPage.jsx

Add a new action:

Record Skill

Teacher flow:

Student Card → Actions → Record Skill

UI flow:

1. select skill type
2. optional note
3. confirm

Phase 2 — Session Experience Improvements

Goal: Improve the **live classroom experience**.

1. Gender-Split Attendance Layout

In:

SessionStartPage.jsx

Display:

Left side → Female students

Right side → Male students

Fallback:

If gender not defined → neutral column.

2. Student Photos

Student cards must display:

avatar

name

status

Sources:

students.photo_url

Fallback:

Cartoon avatar (male/female).

3. Celebration Animation

When student answers correctly:

Add:

- confetti animation
- short applause sound

Suggested libraries:

canvas-confetti

and simple audio playback.

4. Drag & Drop Student Seating

Backend already has:

update_seating_order()

Implement frontend using:

@dnd-kit

Save result to:

class_sessions.seating_order

5. Post-Session Behaviour Recording

After session summary appears, add button:

Add Behaviour

Add Skill

Approve & End

Teacher can still add events before finalizing session.

6. Positive Parent Notifications

When session ends:

Send **positive only notifications** to parents if:

- student answered correctly
- participation high
- skill recorded
- positive behaviour recorded

Notification example:

"Your child participated actively in today's class."

Do not send negative notifications automatically.

Phase 3 — Teacher Dashboard & Student Profile Enhancements

Goal: Improve **teacher visibility and insights**.

1. Add Hijri Date

Dashboard must show:

Gregorian date

Hijri date

Use a Hijri conversion library.

2. Working Hours Indicator

Add indicator:

School Status:

- In Session
- Outside School Hours

Based on school schedule.

3. Teacher Achievements Section

Add a dashboard card:

My Achievements

Metrics:

- sessions conducted
- participation events recorded
- positive behaviours recorded
- skills recorded

Endpoint:

GET /teacher/{id}/achievements

4. Student Profile AI Insights

In teacher view of student profile add:

AI Analysis (Hakim)

Include:

- strengths
- improvement areas
- participation trends
- learning alerts

5. Participation History

Aggregate from:

session_interactions

Display:

- participation count
- trend

- participation rate

6. Parent Information

Teacher must see:

Parent Name

Parent Phone

Parent Contact Button

Data source:

parent_student_relationship

Phase 4 — Reports Intelligence

Goal: Make teacher reports truly useful.

Add AI-driven analysis:

1. Struggling Student Detection

Criteria example:

- attendance < 70%
- participation declining
- homework missing

Highlight students automatically.

2. Teaching Recommendations

Hakim AI should generate suggestions such as:

- increase participation activities
- provide extra exercises
- monitor specific students

3. Behaviour-Linked Messaging

Teacher sending message to parent should allow:

Attach Behaviour Record

Attach Participation Event

Phase 5 — UX Improvements

Goal: polish experience.

Add:

- schedule date picker
- mobile optimization
- real-time updates via WebSocket
- smooth transition animation between session phases

Important Execution Instruction

Implement the work **phase by phase**.

For each phase:

1. complete backend
2. complete frontend
3. connect APIs
4. test full workflow
5. report completion

Reporting Requirement

After finishing each phase, send a report containing:

- files created
- files modified
- new endpoints
- database schema changes
- features completed
- testing results

Final Objective

The final Teacher Module must achieve:

- **100% functional classroom session flow**
- **complete student analytics**
- **AI-assisted teaching insights**
- **stable database-driven architecture**
- **mobile-optimized experience**

You may **start immediately with Phase 1**.